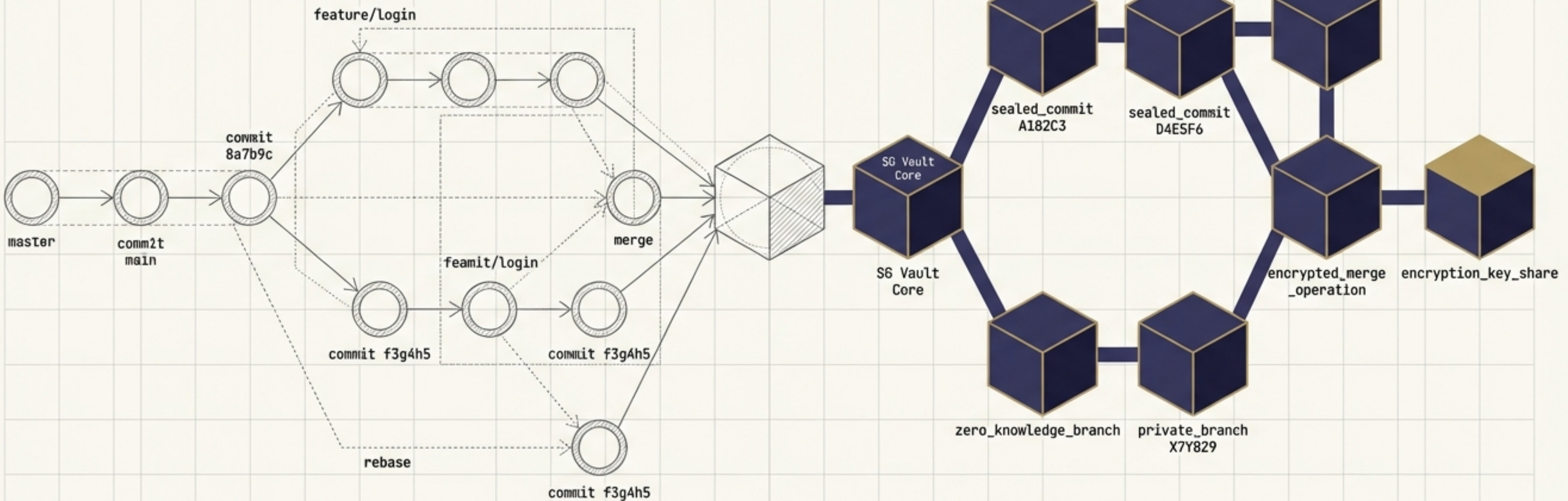


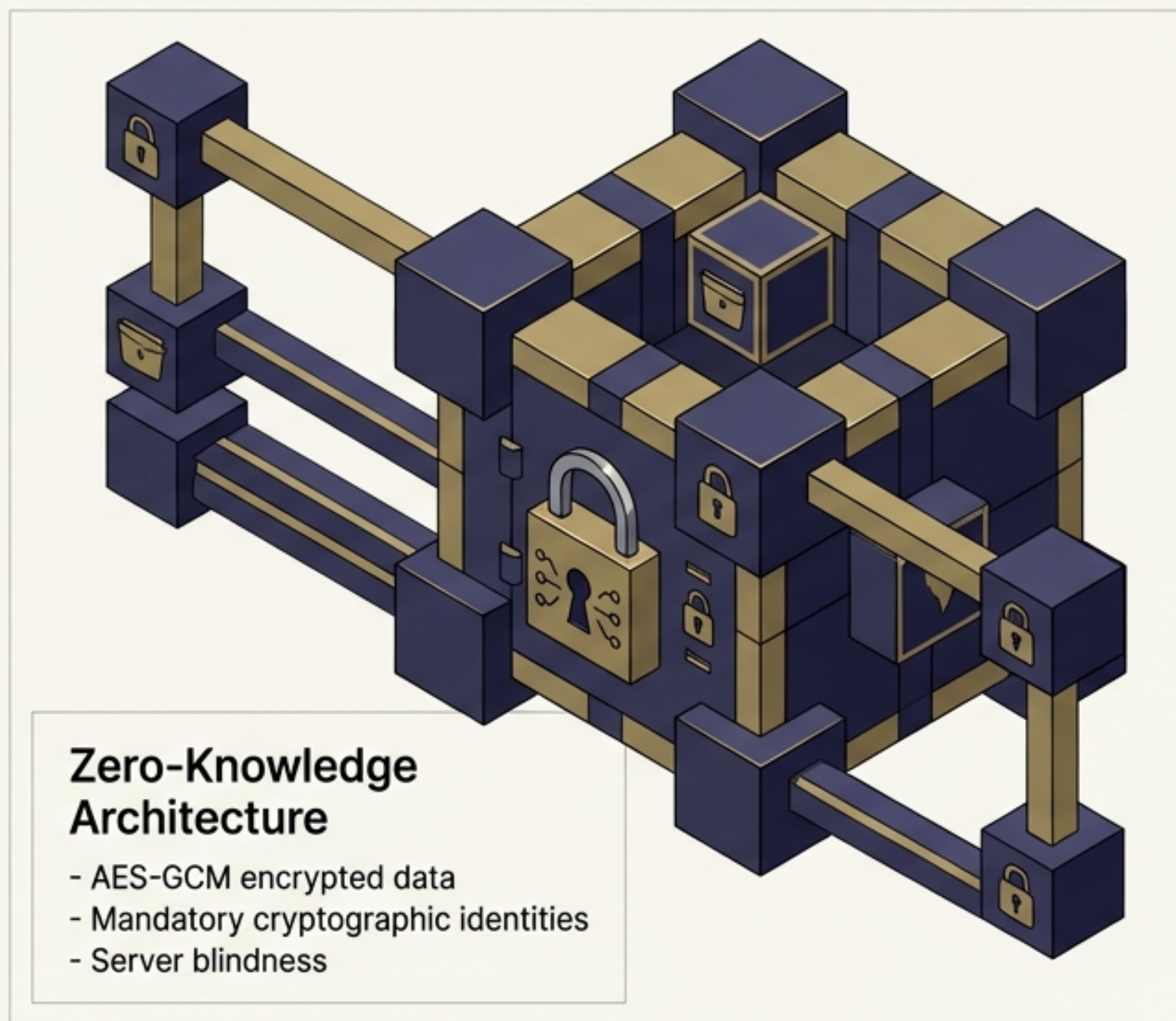
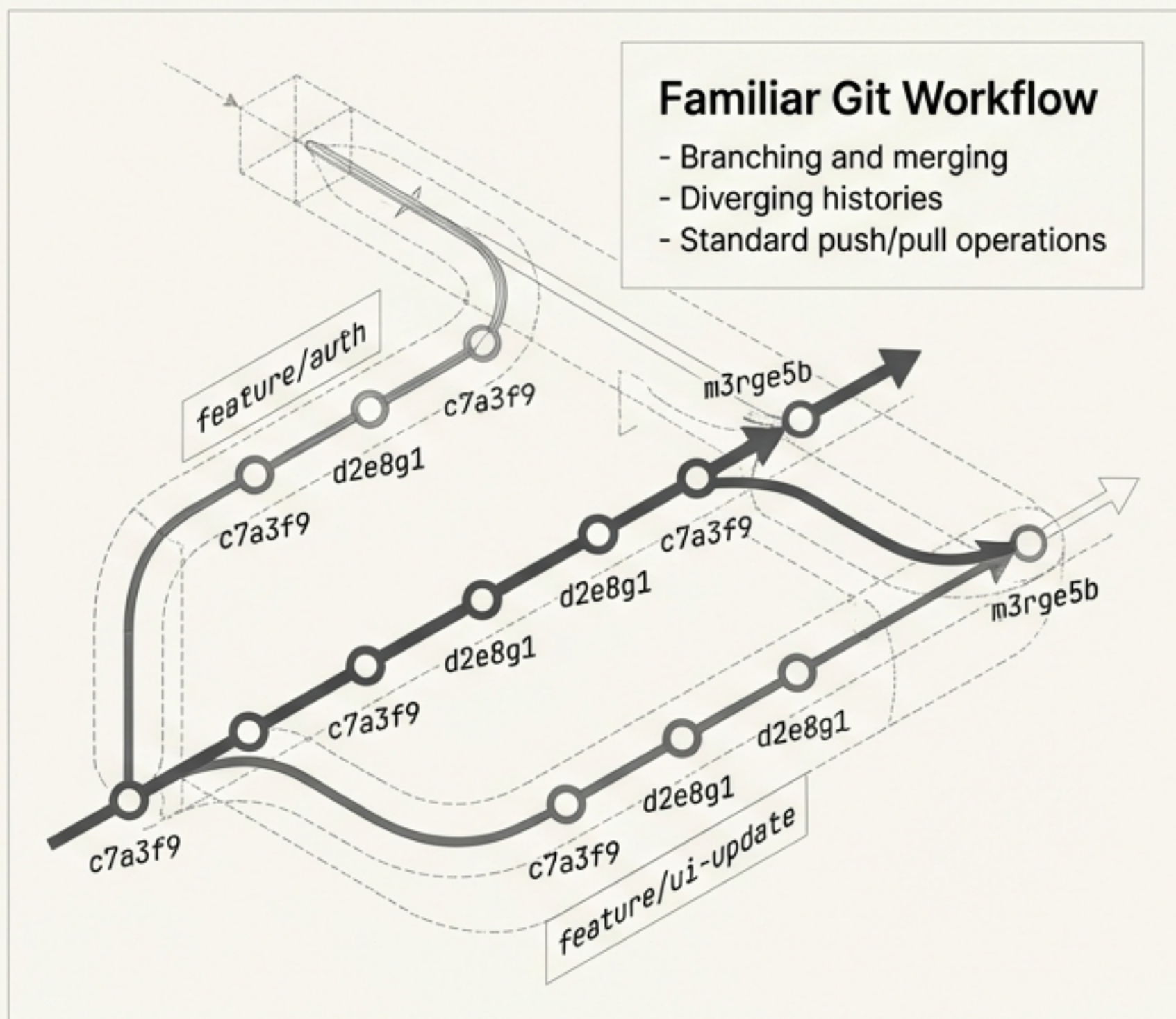
SG Vault vs. Git: The Evolution of Version Control

Mapping the shared foundation and the zero-knowledge divergence.



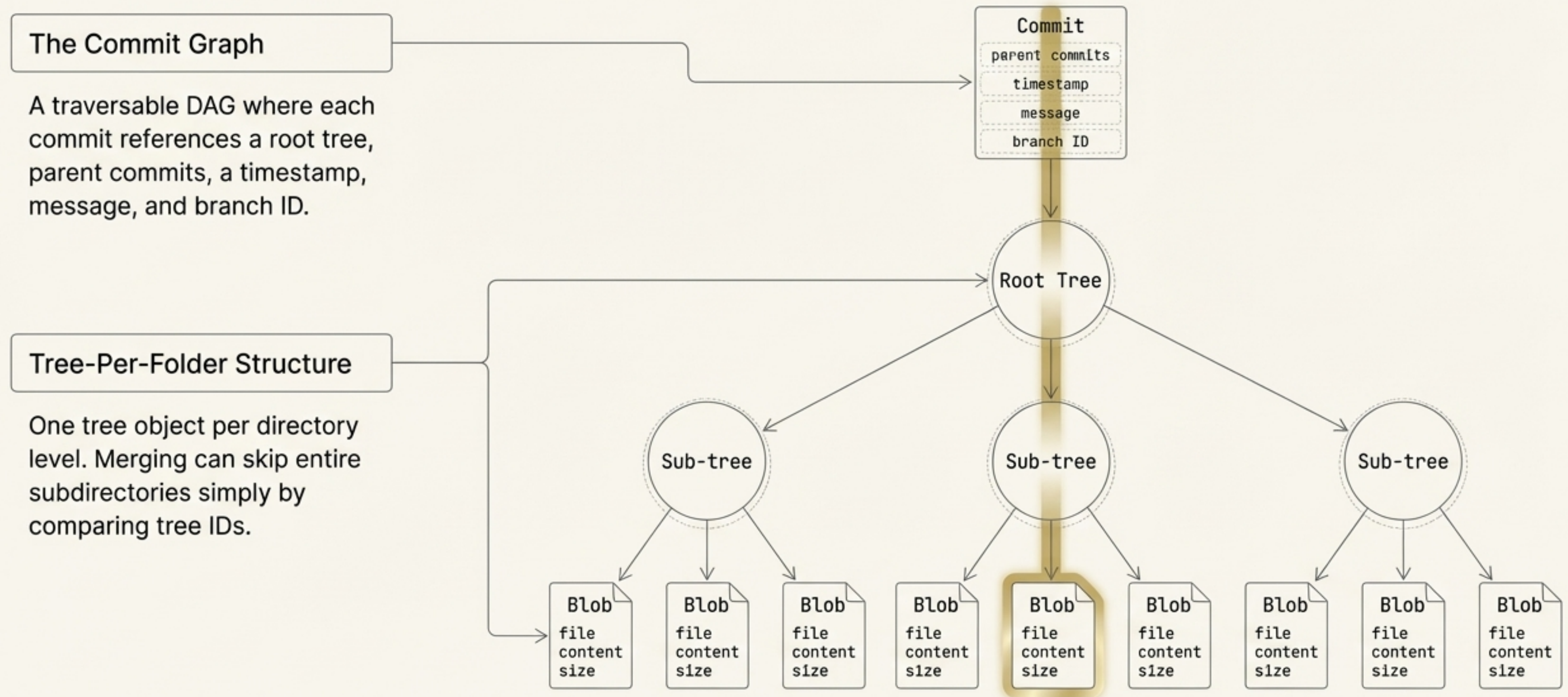
A familiar mental model encased in zero-knowledge security

SG Vault is designed for developers who know Git, but built for an era where the server cannot be trusted. The workflow and data model are intentionally aligned with Git, but the underlying storage completely blinds the host to your code.



The Shared Foundation: What maps 1:1

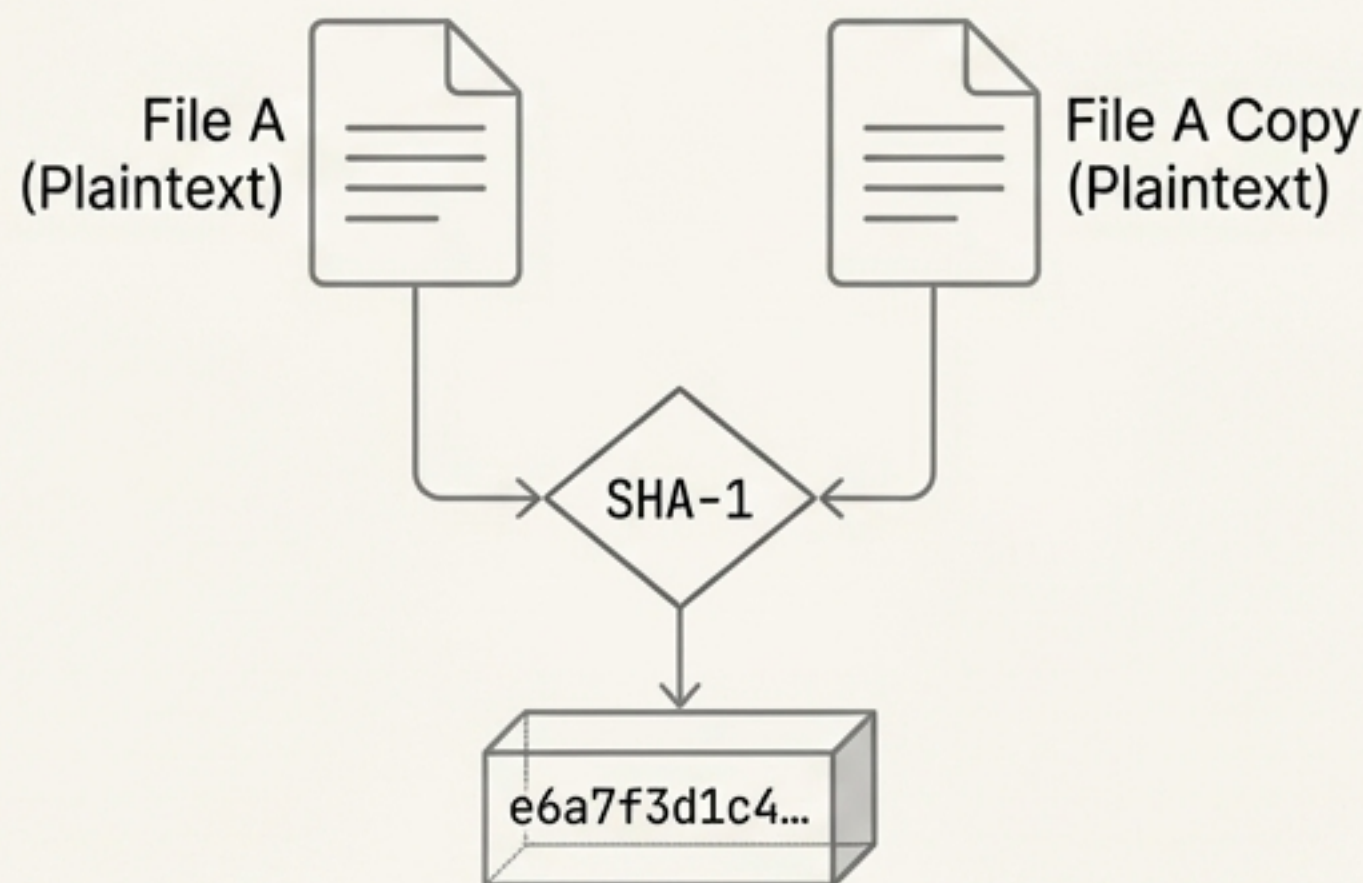
Before data reaches the network, the local representation of an SG Vault repository is architecturally identical to Git.



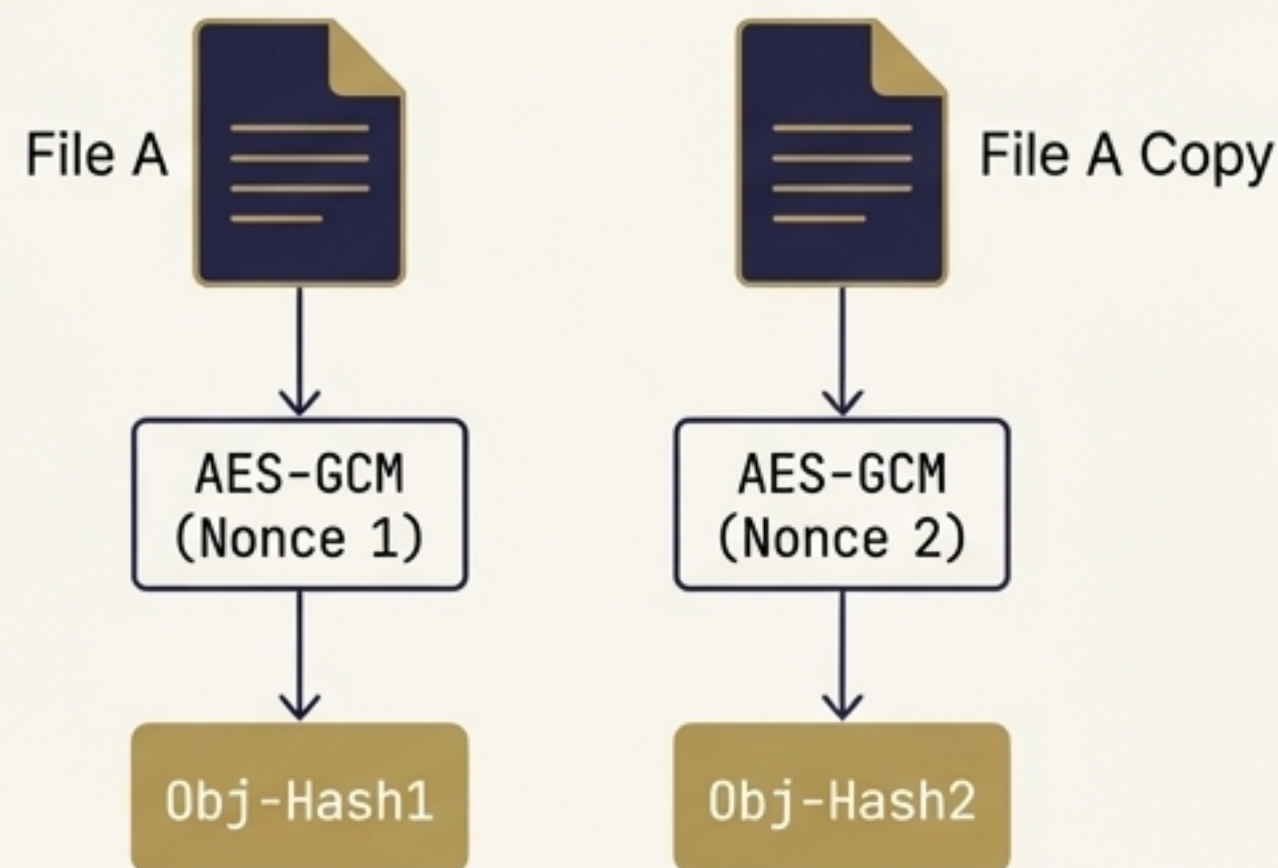
Divergence 1: The Deduplication Trade-off

Git hashes plaintext, resulting in single storage for identical files. SG Vault hashes ciphertext. Sacrificing cross-repo deduplication ensures an attacker cannot infer content equality across different folders.

Git Storage Model



SG Vault Storage Model



Client-Side Change Detection: Tree entries include a `content_hash` field (`SHA256(plaintext)[:12]`). This is encrypted inside the tree entry, invisible to the server, allowing the client to reliably detect edits without re-downloading blobs.

Divergence 2: Complete Server Blindness

SG Vault encrypts all objects (blobs, trees, commits, refs, branch metadata, and keys) with the vault's read_key using AES-GCM before storage. The server sees only opaque blobs.

Git Server View

 src/	Commit Message
 README.md	Initial commit

SG Vault Server View



bare/data/obj-3f9b2



bare/data/obj-7a1c8



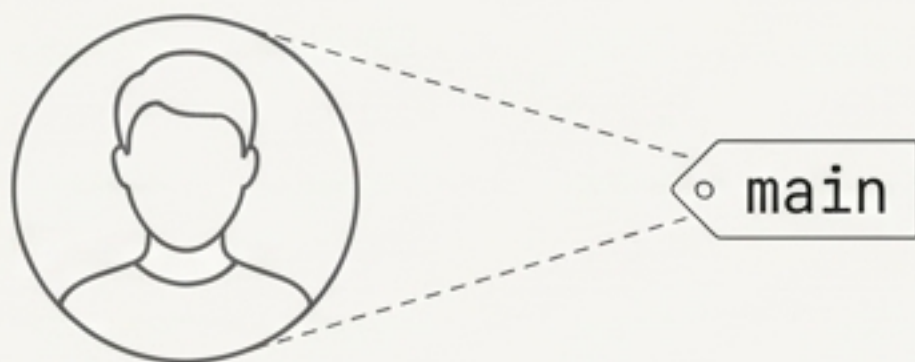
bare/data/obj-9d4e5



Divergence 3 & 4: Cryptographic Identity & Mandatory Signing

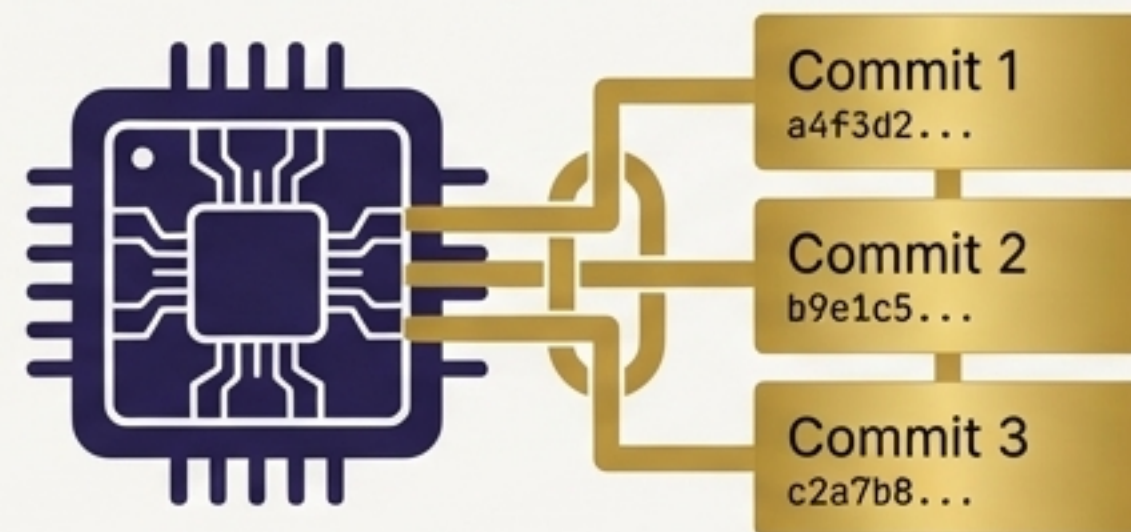
In Git, branches are just movable pointers and commit signing is optional. In SG Vault, the branch is the cryptographic identity, and **unsigned commits cannot exist**.

Git Identity Model



Optional GPG, movable pointers

SG Vault Identity Model



Branch-Per-Clone Identity

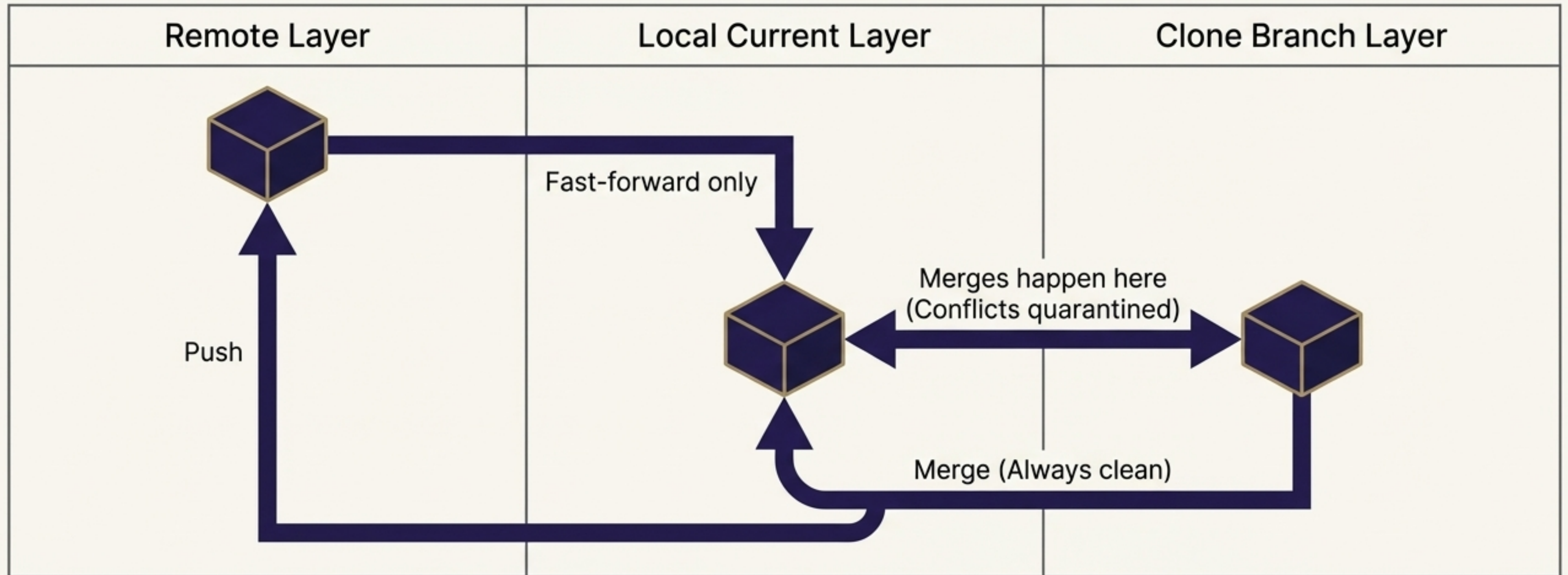
Every `clone()` auto-generates an EC P-256 key pair. The private key never leaves the device.

Mandatory Signatures

Every **commit** is signed by the branch key. The signature locks the `tree_id`, `parents`, `timestamp`, `message`, and **branch_id**. Tampering invalidates the signature.

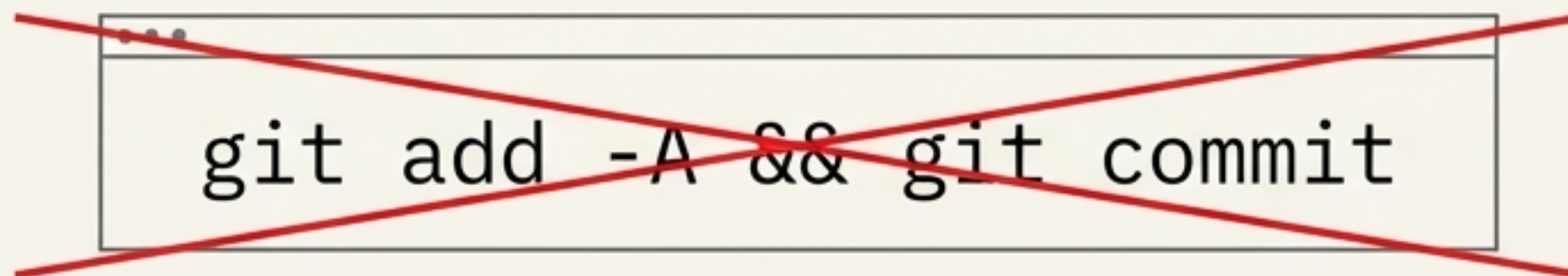
Workflow Shift 1: The Three-Layer Collaboration Model

SG Vault enforces an architectural separation between integration and conflict resolution. The named integration branch never experiences conflicts; your clone branch absorbs them all.

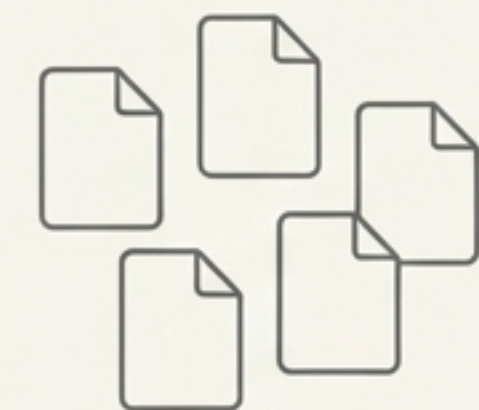


Workflow Shift 2: The Full Snapshot

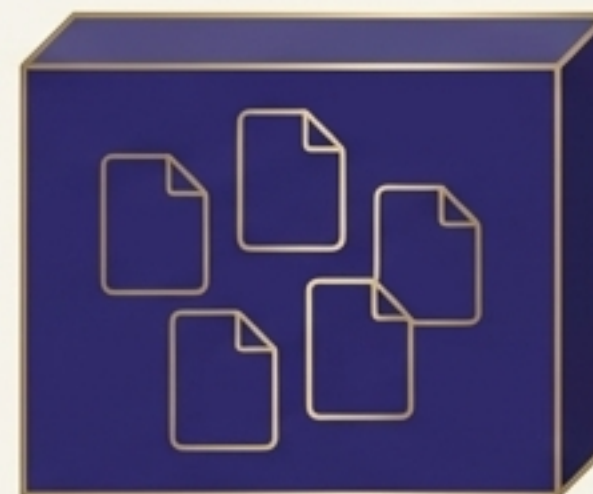
SG Vault has no staging area. A commit always captures the complete state of the working directory.



```
git add -A && git commit
```



Working Directory



Commit Snapshot

Simplified Workflow: Committing in SG Vault always creates a full snapshot of the working directory in a single operation.

Workflow Shift 3: Non-Destructive Conflicts

Git injects conflict markers directly into your code, modifying the files. SG Vault preserves your original code perfectly and **generates a sidecar file for resolution**.

Git Approach: Inline Corruption

```
1  <<<<<<< HEAD
2  function init() {
3      server.start(8080);
4  =====
5  function init() {
6      app.listen(3000);
7  >>>>>>> feature-branch
```

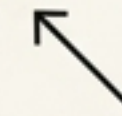
SG Vault Approach: Sidecar Preservation



README.md



.conflict



Resolution Metadata

Resolve the conflict by editing the source file as needed, then simply delete the .conflict file to finalise.

Architectural Nuances: Protecting Metadata

Absolute privacy requires more than encrypting the **file** bodies. **Timestamps** and **file paths** must also be sanitised to prevent **metadata correlation**.

Zero-Knowledge Paths

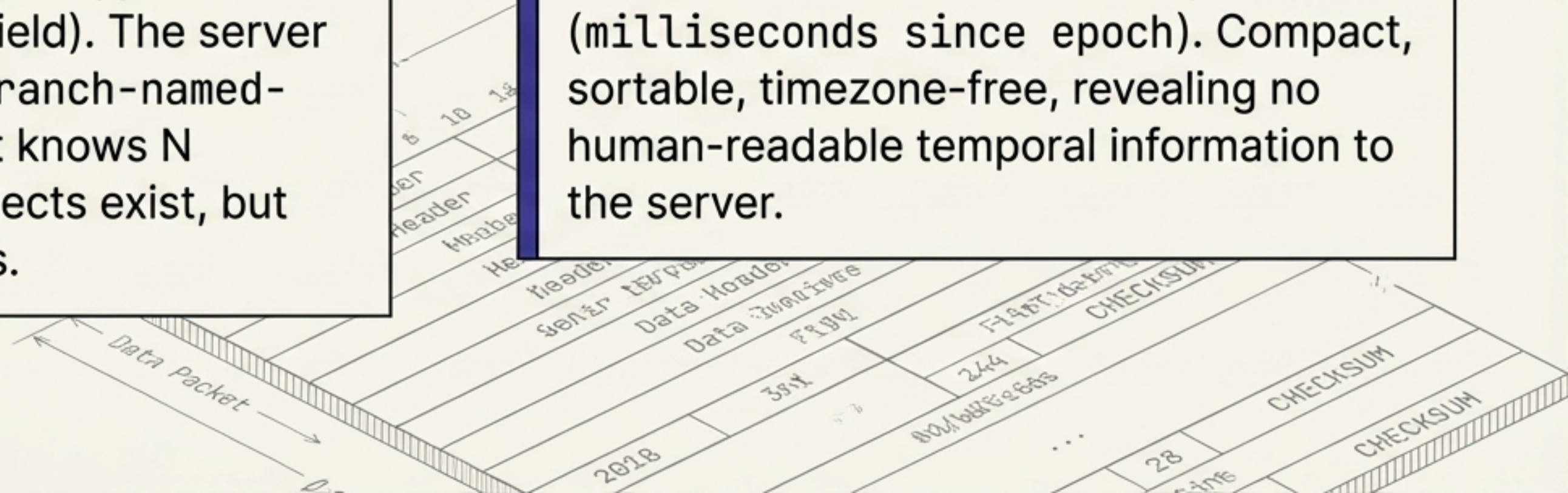
Git: File paths are plaintext in tree objects.

SG Vault: Filenames are encrypted inside tree entries (name_enc field). The server only sees opaque IDs (branch-named-a3f8c2, ref-7b2e1d). It knows N branches and P data objects exist, but nothing of their contents.

Unsigned Integer Timestamps

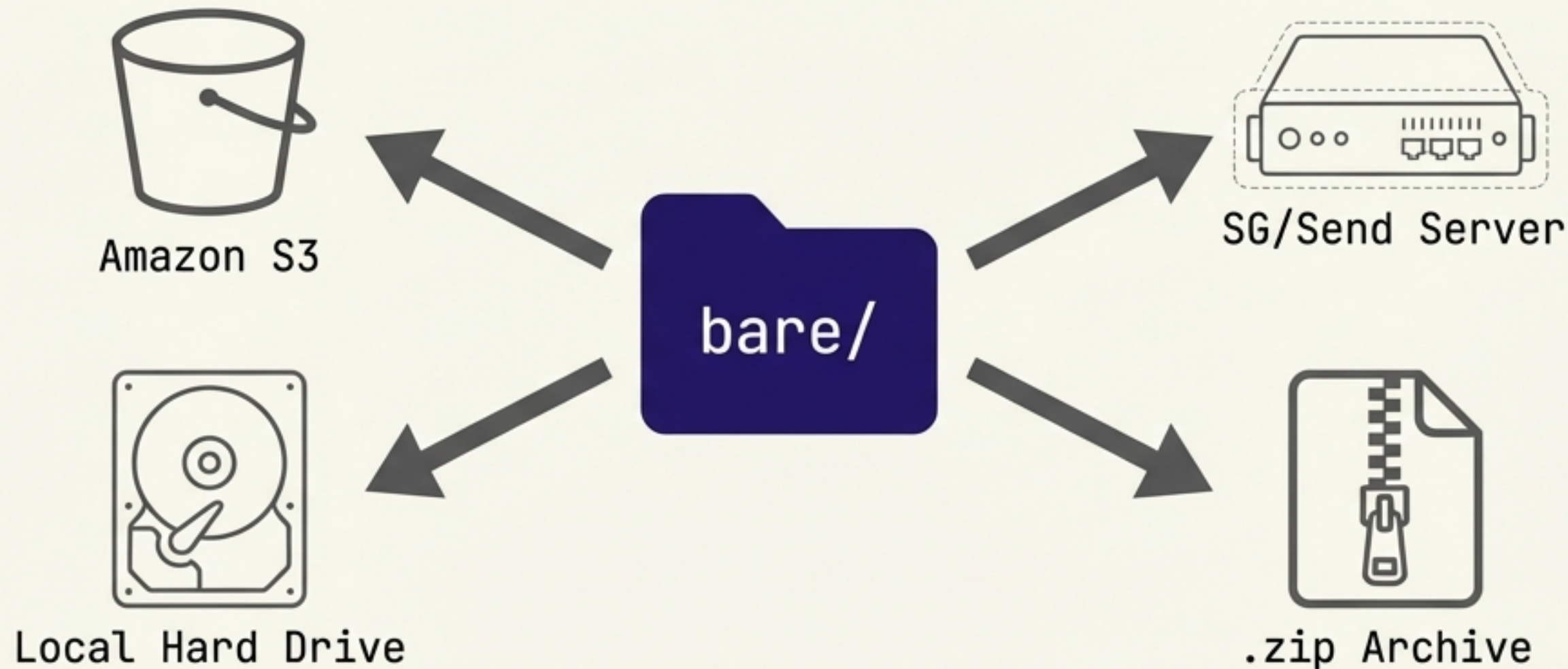
Git: Uses ISO datetime strings.

SG Vault: Uses uint timestamps (milliseconds since epoch). Compact, sortable, timezone-free, revealing no human-readable temporal information to the server.



Infrastructure Shift: The Portable Vault

Because the server only sees opaque data and requires zero repository intelligence, the vault itself is completely **portable**. Delete the server? The local bare/ folder is a perfect backup.



Host Requirements:

The host server only requires four fundamental operations: Read, Write, Delete, and List. This makes the vault infinitely hostable.

Diagnostic Matrix: Data & Security

A rapid reference comparison mapping Git mechanics directly to SG Vault equivalents.

Concept	Git	SG Vault
Object Store	<code>`.git/objects/`</code>	<code>`bare/data/obj-{hash}`</code>
Object Hash Input	Plaintext content	Encrypted ciphertext
Cross-Repo Dedup	Yes (Same content = same hash)	No (Prevents leaking content equality)
Content Encryption	None	<code>`AES-GCM (all objects)`</code>
Commit Signing	Optional GPG	<code>`Mandatory EC P-256`</code>
Filename Visibility	Plaintext in trees	Encrypted in tree entries

Diagnostic Matrix: Workflow & Architecture

Continued reference comparison mapping Git infrastructure and workflow to SG Vault.

Concept	Git	SG Vault
Refs	<code>`.git/refs/heads/main`</code>	<code>bare/refs/ref-{opaque_id}</code>
Branch Metadata	Pointer in <code>.git/refs/</code>	Encrypted file in <code>bare/branches/</code>
Tree Granularity	One per folder	One per folder (Identical)
Staging Area	<code>git add</code>	None (Full-snapshot commits)
Conflict Handling	Inline markers in code	Non-destructive <code>.conflict</code> files
Server Intelligence	Full repo access	Dumb blob store
Remote Backends	Git protocol, HTTP	S3, SG/Send, local folder, zip



Familiarity encoded in absolute privacy

The usability, speed, and mental model of Git, architected from the ground up for a zero-trust world. The code is yours. The history is yours. The server sees nothing.